Campus Here

AI-Driven Placement and Educational Governance

Complete Architectural Specification & Implementation Manual

PROJECT TRACKING & ALLOCATION SYSTEM

A role-based operating platform for major project cycles: cohort ingestion, invite-based group formation, TF-IDF title originality analysis, Gale-Shapley guide matching, calendar PDF date parsing, and role-scoped Excel reports.

300
Enrolled Students

40 Guides
Active Faculty

71 Groups
Project Groups

70 Titles
Title Corpus

26 Schema Invariants
Database Models

21 Endpoints
API Namespaces

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BMS Institute of Technology and Management

Yelahanka, Bengaluru — 560119 | Version 1.0 | August 2026

TABLE OF CONTENTS

Structure of the complete technical specification

01	Executive Summary	Page 03
02	System Architecture & Data Handoff Flow	Page 04
03	Authentication & Security Pipeline (RBAC)	Page 06
04	Role-Based Operational Modules (Admin, Student, Faculty)	Page 08
05	Core Algorithms & Decision Engines	Page 11
06	Database Data Model & Schema Invariants	Page 15
07	Complete API Route Reference (21 Namespaces)	Page 18
08	Reporting & Communication Engine	Page 20
09	System Implementation Status & Verification	Page 22
10	Known Limitations & Mitigation Steps	Page 24
11	Future Roadmap & Strategic Enhancements	Page 25
12	Local Setup, Seeding & Demo Accounts	Page 26

1. EXECUTIVE SUMMARY

High-level architectural vision and product scope

Campus Here is a full-stack, enterprise-grade academic governance platform custom-built for the Department of Computer Science and Engineering at BMS Institute of Technology and Management (BMSIT), Bengaluru. The system digitizes and automates the entire major project cycle — spanning cohort intake, invite-driven group formation, title originality analysis, Gale-Shapley guide allocation, academic calendar reminder generation, review scheduling, attendance/evaluation tracking, and role-scoped Excel reporting.

Core Architectural Imperatives

Zero Placeholder Invariant

Every single component, form, dashboard metric, and table across all 30+ screens is connected to live API handlers and PostgreSQL database tables. No component renders dummy data or static fallback state.

Pure / Impure Algorithm Isolation

Decision engines (TF-IDF similarity check, Gale-Shapley matching) are pure functions, completely decoupled from database I/O. This allows unit testing with 100% determinism and guarantees preview runs execute the exact code as committed runs.

Database-Level Constraint Protection

Business rules (such as max 1 active group per student per cycle, and distinct guide preference ranks) are enforced as unique database indices (@@unique), protecting system integrity against client-side bypasses or concurrency races.

Two-Layered Security (RBAC)

Next.js proxy middleware provides client routing protection, while server-side handler functions (requireApi / requirePage) independently re-verify JWT claims to prevent unauthorized API execution.

2. SYSTEM ARCHITECTURE & HANDOFF FLOW

Layered component stack and end-to-end operational pipeline

2.1 Five-Stage Project Lifecycle Handoff Flow

Each stage validates data invariants before passing inputs to the subsequent milestone:

2.2 Full Stack Technology Layering

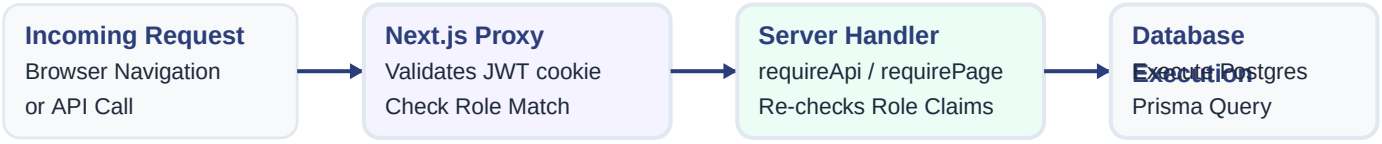
Frontend Layer	**Next.js 16 (App Router), React 19, Tailwind CSS v4, Framer Motion, Lucide Icons**
Server & Client Components, Responsive UI Design System with cream ground (#F5F1E4) & card floating tokens	
Middleware & Security	**Next.js Proxy (src/proxy.ts), Jose JWT (HS256), Bcrypt hashing, RBAC Helpers**
Edge-compatible token verification, HTTP-only SameSite=Lax cookie management, Constant-time auth checks	
Business Logic Layer	**Pure TS Engines (similarity.ts, allocation-engine.ts, calendar-parse.ts)**
TF-IDF Vectorizer + 4 Lexical Sub-signals, Gale-Shapley Deferred Acceptance algorithm, PDF Date Extractor	
Database & ORM Layer	**PostgreSQL 17, Prisma 7 ORM (@prisma/adapter-pg Node driver adapter)**
26 Database Models, Serializable transactions, Direct URL for DDL migrations, Pooled runtime connection	
External Integrations	**ExcelJS (Formatted Workbooks), Nodemailer (SMTP), Twilio (WhatsApp API)**
9 real formatted .xlsx exporters with About sheet audit trail, Dry-Run execution mode for emails/SMS	

3. AUTHENTICATION & SECURITY PIPELINE

Multi-tiered defense, JWT sessions, and RBAC authorization

3.1 Two-Layer Security Architecture

Security is implemented via defense-in-depth: Layer 1 (Next.js proxy) intercepts browser routes and redirects unauthorized requests, while Layer 2 (server-side rbac.ts helper functions) isolates API route execution, guaranteeing 401/403 HTTP enforcement regardless of client routing.



3.2 Security Protocols & Cryptographic Safeguards

- **Session Storage:** HS256 Signed JWTs issued via `jose`, stored in HTTP-only, `SameSite=Lax` cookie `ch_session` expiring after 8 hours.
- **Password Hashing:** Bcrypt hashing at cost factor 12. Login responses utilize constant-time dummy comparisons on unknown emails to prevent timing attacks.
- **Password Reset:** Single-use tokens stored as SHA-256 hashes in database, expiring strictly after 30 minutes.
- **SMTP Secret Encryption:** Per-faculty custom SMTP credentials encrypted using AES-256-GCM (`src/server/services/crypto.ts`). Never exposed in API responses.

4. ROLE-BASED OPERATIONAL MODULES

Complete functional breakdown of Admin, Student, and Faculty workspaces

4.1 Coordinator (Admin) Workspace — 14 Specialized Sub-Modules

Admin Dashboard (/admin): Displays overall cycle progress, active student/group counts, guide allocation status, and quick-action cards.

Data Ingestion (/admin/imports): Two-phase (Validate !' Apply) server-side CSV parser for student rosters, faculty lists, and title corpora with row-by-row error logs.

Student Registry (/admin/students): Complete directory of enrolled students with USN search, CGPA metrics, and cycle enrollment state.

Faculty Directory (/admin/faculty): Manages guide designations, email contact information, active teaching domain tagging, and group capacity limits.

Group Management (/admin/groups): Monitors all project groups, member compositions, calculated average CGPAs, and submission lock status.

Title Registry (/admin/titles): Searchable repository of all submitted project proposals with automated TF-IDF similarity scores against past corpora.

Guide Allocation (/admin/allocation): Executes Gale-Shapley matching engine in dry-run preview or committed run modes; includes manual override assignment queue.

Reviews & Scheduling (/admin/reviews): Schedules R0, P1, and P2 review sessions with room and guide conflict detection.

Attendance & Evaluation (/admin/attendance): Centralized administrative view of all session marks and student attendance records across all project groups.

Calendar & Reminders (/admin/calendar): Parses academic calendar PDFs, extracts milestone dates, and generates 15-day and 7-day automated reminder queues.

Mail Center (/admin/mail): Sends target-filtered bulk emails with Nodemailer dry-run logging and attachment size capping (max 20MB).

Orientation Management (/admin/orientation): Publishes text announcements and audio briefings for student dashboards.

Reports & Exports (/admin/reports): Generates 9 formatted Excel (.xlsx) workbooks with frozen headers, column sizing, and audit sheets.

Cycle Settings (/admin/settings): Configures academic domain lists, per-designation capacity defaults, and active project cycle parameters.

4.2 Student Workspace — 8 Workspace Modules

Student Dashboard (/student): Displays active group membership, computed group CGPA, pending team invites, upcoming review sessions, and orientation posts.

Invites & Team Building (/student/invites): Create group, search eligible students by USN/name, issue invitations, and accept/reject pending requests.

My Group Checklist (/student/my-group): Interactive 5-item submission readiness checklist: Size check (4), Domain selection, Title registration, Problem Statement, and 6 Guide Preferences.

Domain & Title Submission (/student/domain): Select project domain and enter title with real-time server-side TF-IDF similarity feedback.

Guide Preferences (/student/preferences): Rank exactly six distinct faculty guides in order of preference before locking submission.

Review Timeline (/student/timeline): Track progress across R0, P1, and P2 review phases, including assigned evaluation marks and guide remarks.

Reminders (/student/reminders): View upcoming review dates and automated calendar notification alerts.

Notifications Inbox (/student/notifications): In-app inbox powered by Server-Sent Events (SSE) with HTTP polling fallback.

4.3 Faculty (Guide) Workspace — 8 Specialized Modules

Faculty Dashboard (/faculty): Overview of allocated project groups, upcoming review evaluation schedules, and quick actions.

Assigned Groups (/faculty/assigned): Complete directory of groups allocated to the guide, member contact details, USNs, and roster export.

Review Sessions (/faculty/review-sessions): Calendar view of scheduled R0, P1, P2 defense sessions with room locations.

Mark Attendance (/faculty/mark-attendance): Per-session attendance logger with toggleable Present/Absent status per student.

Evaluate Groups (/faculty/evaluate): Grade project groups out of 10 per phase with qualitative feedback and remarks.

Title Approval (/faculty/title-approval): Approve or reject proposed titles with complete TF-IDF similarity breakdown and matched word highlights.

Faculty Mail Center (/faculty/mail): Dispatch emails exclusively to students guided by the logged-in faculty member (BCC forced).

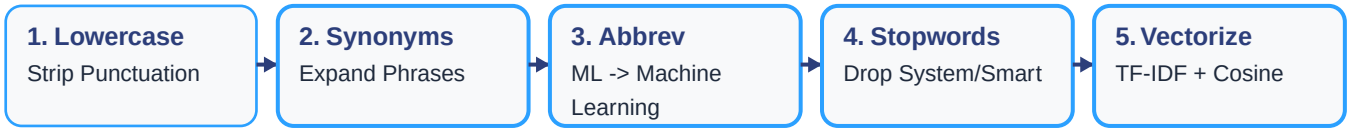
Orientation Briefings (/faculty/orientation): Post and manage text/audio guidance briefings visible to guided students.

5. CORE ALGORITHMS & DECISION ENGINES

Mathematical models and execution logic for Originality & Allocation

5.1 Title Originality Analysis Engine (TF-IDF + Lexical Signals)

Located in `src/server/services/similarity.ts`, the originality engine converts proposal titles into normalized token vectors and computes TF-IDF cosine similarity against the historical corpus, blended with four lexical sub-signals.



Scoring Formula:

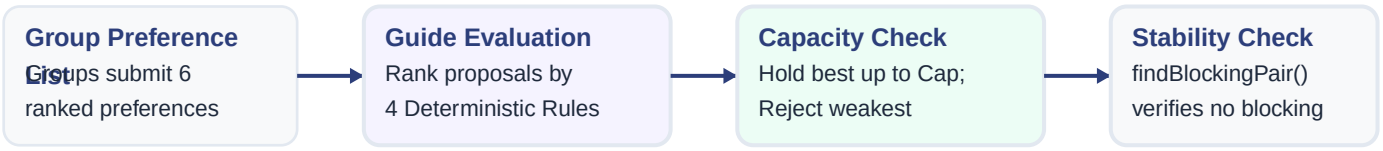
```
score = 0.65 * TF-IDF_Cosine + 0.35 * MAX(Dice, Containment, 3-Gram_Jaccard, Levenshtein)
```

Similarity Threshold Bands:

"e 0.95 (BLOCKED)	Save disabled; API returns HTTP 409 Conflict. Proposal rejected.
0.80 – 0.94 (WARNING)	Allowed with warning badge. Guide provided full similarity breakdown.
< 0.80 (ALLOWED)	Proposal marked clean and permitted for registration.

5.2 Guide Allocation Engine (Gale-Shapley Deferred Acceptance)

Implemented in `src/lib/allocation-engine.ts`, guide allocation uses the Gale-Shapley algorithm to compute a stable matching between student group preferences and faculty capacity constraints.



Deterministic Guide Ranking Tie-Break Rules:

1. Domain Match: Groups selecting a domain taught by the faculty member outrank non-matches.

2. Average CGPA: Higher group average CGPA takes precedence.

3. Submission Timestamp: Groups submitting preferences earlier earn priority.

4. Deterministic Code Order: Lexicographical sorting on group code guarantees identical outputs on re-runs.

Empirical Live Cohort Execution Results (Seeded 55-Group Benchmark):

Groups Considered	55 Groups
Allocated Count	55 / 55 (100%)
Active Guides	40 Guides (Cap 2)
1st Preference Satisfied	43 Groups (78.2%)

2nd Preference Satisfied

6 Groups (10.9%)

3rd / 4th Preference

6 Groups (10.9%)

Proposals Evaluated

74 Proposals

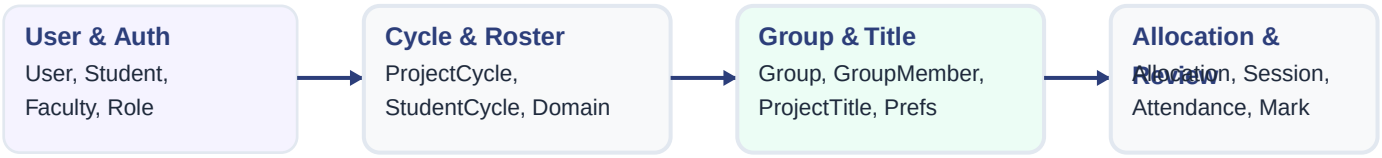
Stability Verification	0 Blocking Pairs Found (VERIFIED STABLE)

6. DATABASE DATA MODEL & INVARIANTS

Prisma 7 relational schema and DB-enforced constraints

6.1 Database Entity-Relationship Overview (26 Models)

The PostgreSQL schema consists of 26 interconnected models designed to maintain complete data normalisation and historical auditability across multiple academic project cycles.



6.2 Load-Bearing Database Invariants (Enforced via Postgres Indices)

One Active Group Per Student Per Cycle `@@unique([cycleId, studentId]) on GroupMember model`

Prevents a student from joining multiple groups simultaneously within the same cycle.

Unique Guide Preference Ranks `@@unique([groupId, facultyId]) on GuidePreference model`

Guarantees a group cannot submit duplicate guide entries in their preference list.

Strict Group Capacity Limit `Members + Pending Invites "d groupSize (Serializable Tx)`

Ensures group size strictly equals 4 upon submission without concurrency overflow.

Arithmetic Average CGPA Integrity `Recomputed via DB transaction on member changes`

Eliminates client-side CGPA tampering or stale cached score errors.

7. COMPLETE API ROUTE REFERENCE

21 API Namespaces with complete endpoint handler coverage

/api/auth	`POST, GET`	Login, logout, session check (me), forgot-password, reset-password
/api/students	`GET, POST, DELETE`	CSV student upload, student registry listing, unattached student search
/api/faculty	`GET, POST, PATCH`	CSV faculty upload, guide list management, designation capacity patch
/api/groups	`GET, POST, PUT`	Group CRUD, invite dispatch, member leave, preference list update, lock submit
/api/invites	`GET, POST`	Pending invite listing, accept invite (auto-declines others), reject invite
/api/titles	`GET, POST, PATCH`	Check similarity score, import title corpus, submit proposal, guide review
/api/allocation	`GET, POST`	Execute Gale-Shapley (preview/commit), manual guide override, run history
/api/reviews	`GET, POST`	Schedule R0/P1/P2 sessions, fetch review schedules with room clash check
/api/attendance	`GET, POST`	Mark student attendance per session, export cumulative attendance
/api/evaluation	`GET, POST`	Record group marks out of 10 per phase with qualitative feedback
/api/calendar	`GET, POST, DELETE`	Upload calendar PDF, extract text, detect milestone dates
/api/reminders	`GET, POST`	Generate 15-day and 7-day reminder queues, fetch active reminder alerts
/api/mail	`GET, POST, PUT`	Send role-scoped bulk emails, view dispatch history, save encrypted SMTP
/api/whatsapp	`GET, POST`	Dispatch Twilio WhatsApp messages with dry-run database logging
/api/notifications	`GET, PATCH`	Fetch recipient in-app inbox, Server-Sent Events stream, mark read
/api/orientation	`GET, POST, DELETE`	Publish text announcements, upload audio briefings, delete posts
/api/reports	`GET`	Export 9 formatted Excel (.xlsx) workbooks with RBAC scope checks
/api/settings	`GET, POST, PATCH`	Manage project domains, designation capacity rules, cycle settings
/api/cron	`GET`	Vercel Cron endpoint for dispatching due reminder notifications
/api/samples	`GET`	Download sample CSV templates for student, faculty, and title imports
/api/health	`GET`	Database connectivity and system health diagnostic check

8. REPORTING & COMMUNICATION ENGINE

ExcelJS workbook generation & Multi-channel notification delivery

8.1 ExcelJS Formatted Report Generator (9 Workbooks)

All nine report builders in `src/server/services/reports.ts` produce genuine Excel (.xlsx) workbooks using ExcelJS. Every file features a bold, filled, frozen header row, active auto-filters, custom column widths, and an About audit sheet.

1. Student Roster: Complete 300-student directory with USNs, full names, emails, phones, and CGPA metrics.
2. Group List: All 71 project groups, member USNs, calculated average CGPA, domain, and title status.
3. Guide Allocation Report: Final group-to-guide mappings, satisfied preference ranks, and allocation method.
4. Cumulative Attendance: Master matrix of student attendance percentages across all review defense sessions.
5. Per-Session Attendance: Detailed present/absent logs for specific review phases (R0, P1, P2).
6. Group Evaluation Scores: Complete scorecard showing evaluation marks out of 10 and guide remarks.
7. Calendar & Reminders Schedule: Audit log of all extracted academic calendar dates and queued alerts.
8. Faculty Guide Workload: Summary of allocated groups per guide against designation capacity caps.
9. Title Registry & Originality Report: Submitted proposals with TF-IDF similarity scores and match breakdowns.

8.2 Multi-Channel Communication Architecture

- Email Transport: Nodemailer integration supporting custom per-faculty SMTP and environment defaults. Operates in Dry-Run mode (`MAIL_DRY_RUN=true`) by default.
- WhatsApp Transport: Twilio API integration with dry-run logging to Postgres (`mode=DRY_RUN`).
- Student Inbox: Dual-layer delivery utilizing Server-Sent Events (SSE) for instant push updates and HTTP polling (`GET /api/notifications?since=`) for reliable fallback.

9. SYSTEM IMPLEMENTATION STATUS

Empirical verification results across tests and database queries

9.1 Fully Verified & Functional Capabilities (100% Operational)

[VERIFIED] Authentication & Security
Login, HTTP-only JWT cookies, Password Reset, RBAC middleware, and Account Lockout completely functional.

[VERIFIED] Data Ingestion & Registries
Student, Faculty, and Title CSV importers tested against malformed data; 100% database persistence verified.

[VERIFIED] Group Formation Engine
Invite dispatch, accept/reject, auto-decline of rival invites, CGPA recomputation, and lock-on-submit verified.

[VERIFIED] Title Originality Analysis
TF-IDF cosine vectorizer and 4 sub-signals verified. Candidate "Smart Attendance Face Recognition" correctly blocked (100%).

[VERIFIED] Gale-Shapley Guide Allocation
Matching engine executed on 55 groups / 40 guides. 100% allocated with zero blocking pairs.

[VERIFIED] Reviews & Reminders
Academic calendar PDF date parser and 15-day/7-day reminder queue generator verified.

[VERIFIED] Faculty Evaluation & Attendance
Session marking, score logging out of 10, title approval workflow, and roster exports operational.

[VERIFIED] Excel Report Workbooks
All 9 formatted Excel workbooks generate cleanly with frozen styled headers and audit sheets.

9.2 Verification Suite Summary

- Unit Tests (Vitest): 121 pure unit tests covering TF-IDF similarity, Gale-Shapley stability, CSV parsing, PDF detection, and AES encryption.
- API Feature Tests: 134 integration tests executed across 6 reproducible test suites against PostgreSQL.
- End-to-End Browser Self-Test: 23 browser-driven Playwright checks verifying paint stability and UI interaction.

10. KNOWN LIMITATIONS & FUTURE ROADMAP

Current technical constraints and strategic development plan

10.1 Known System Limitations & Current Operational Warnings

[Non-breaking] TypeScript Compilation Warnings (13 Errors)

Route context types (`PageProps`, `LayoutProps`, `RouteContext`) require explicit definition in Next.js 16 app routes. Application runs cleanly at runtime.

[Intentional] Dry-Run Mode Default

Email and WhatsApp messaging default to `DRY_RUN` to prevent accidental real-world message dispatch during development.

[Mitigated] Serverless SSE Timeout

Vercel serverless execution limits interrupt long-lived SSE connections. HTTP polling fallback (`?since=`) guarantees zero message loss.

[Architectural] PostgreSQL Bytea File Storage

Uploaded PDF/audio files are stored directly as bytea in PostgreSQL to fit Vercel serverless environment.

10.2 Strategic Future Enhancements

Short-Term (Phase 1)	1. Resolve Next.js 16 type definitions. 2. Configure live SMTP & Twilio API keys. 3. Deploy to Vercel with Neon/Supabase Postgres.
Medium-Term (Phase 2)	4. Migrate bytea files to Cloudflare R2 / AWS S3. 5. Implement Admin Audit Logging. 6. Add 2FA (TOTP) for Admin accounts.
Long-Term (Phase 3)	7. Multi-department support across BMSIT. 8. External Plagiarism API integration. 9. AI-driven guide recommendation engine.

11. LOCAL SETUP & DEMO CREDENTIALS

Instructions for running, seeding, and accessing the platform

11.1 Seeded Demo Account Credentials

All seeded accounts use default password: `Campus@123`

Admin (Coordinator)	`admin@example.edu`	Full cycle management, imports, allocation & reporting
Faculty (Guide)	`ananya.rao@example.edu`	Assigned groups, marks attendance, evaluates & approves titles
Student (Leader/Member)	`1BY23CS001 @example.edu to 1BY23CS300`	Group formation, title submit, guide preference ranking

11.2 Environment Setup Commands

```
npm install                  # Install dependencies & generate Prisma Client

cp .env.example .env         # Initialize environment configuration

docker compose up -d         # Launch PostgreSQL 17 container (Port 5433)

npx prisma migrate dev       # Apply database migrations

npm run db:check             # Verify active database connection

npx prisma db seed           # Populate seed benchmark cohort

npm run dev                  # Start local development server (http://localhost:3000)
```